LAB EXPERIMENT REPORT

Application & Software Development / DevOps Practicum

Name	: Heenal Dhanwani	**Roll No.**	: 2401024
Subject	: ASD&D	**Experiment**	: 05

PART A — Continuous Integration & Jenkins Installation

Aim: To understand Continuous Integration and Jenkins installation.

LO Mapped: LO3

1.1 Theory

Continuous Integration is a development practice that promotes collaboration, early issue detection, and automated testing to enhance the quality and efficiency of software development.

Jenkins is an open-source automation server that facilitates continuous integration and continuous delivery (CI/CD) of software projects. It helps developers automate various stages of the software development lifecycle, from building and testing to deployment and monitoring. Jenkins supports a wide range of plugins and integrations, making it a versatile tool for automating various tasks in software development.

1.2 Steps to Install Jenkins

I. Java Installation: Ensure Java Development Kit (JDK) is installed on your system.



Fig. 1 — Verifying the Java installation (java --version)

II. Download Jenkins: Get the latest Jenkins WAR file from the official website or package manager.



Fig. 2 — Installing the generic .war file

III. Launch Terminal: Open a terminal/command prompt on your system. command: java –jar C:\Users\Asus\Downloads\jenkins.war —-enable–future–java



Fig. 3 — Running the above command in the terminal

IV. Navigate to Directory: Use the cd command to navigate to the directory containing the Jenkins WAR file.



Fig. 4 — Copying the initial admin password and directory path

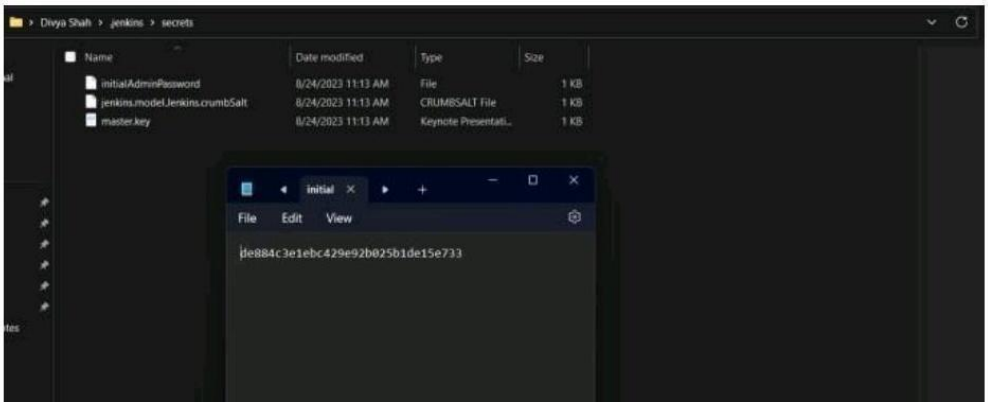


Fig. 5 — Directory contents: password and key files

V. Start Jenkins: Run the command java -jar jenkins.war to start the Jenkins server.

VI. Access Web UI: Open a web browser and go to http://localhost:8080 to access the Jenkins setup wizard.

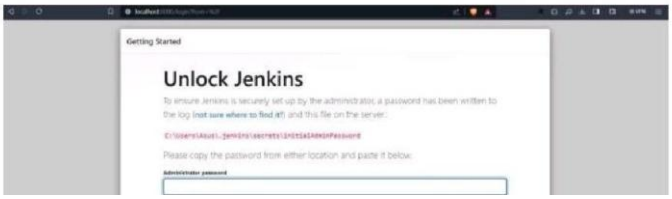


Fig. 6(a) — Navigate to "localhost:8080" and enter the password



Fig. 6(b) — Click "Install suggested plugins"

VII. Unlock Jenkins: Retrieve the initial admin password from the terminal and enter it in the setup wizard.

VIII. Customize Plugins: Choose recommended or custom plugins to install during setup.

IX. Create Admin User: Set up an admin user account with credentials.

X. Jenkins URL: Confirm the Jenkins URL and complete the setup process.

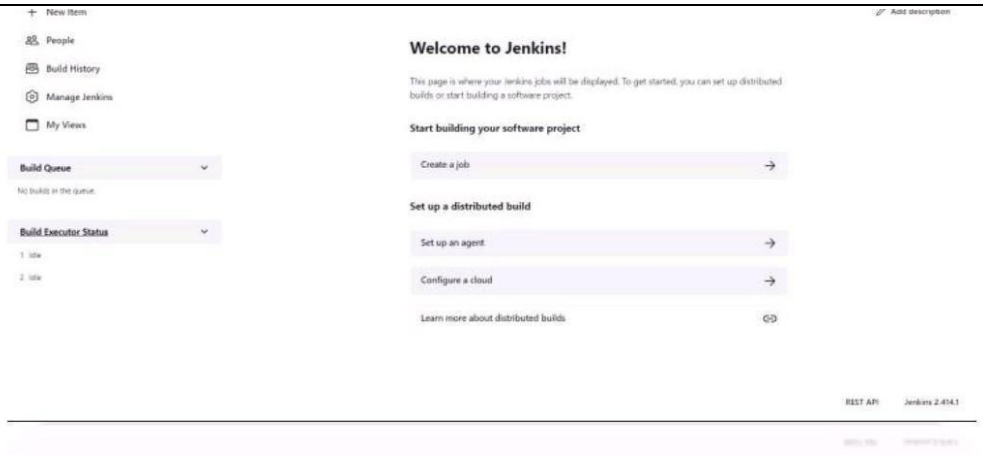


Fig. 7 — Jenkins dashboard after successful plugin installation and setup XI.

Dashboard: Log in with the admin credentials to access the Jenkins dashboard.



Fig. 8 — Signing in with the admin username and password

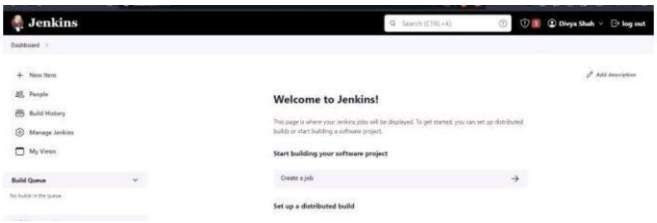


Fig. 9 — Jenkins dashboard, ready for use

CONCLUSION

By this assignment we learned about Continuous Integration and the way to install Jenkins.

PART B — Building a Pipeline using Jenkins

Aim: To build a pipeline using Jenkins.

2.1 Theory

What is a Pipeline in Jenkins?

A Jenkins Pipeline is a set of automated processes that help developers and DevOps teams build, test, and deploy code in a continuous integration/continuous delivery (CI/CD) environment. In simple terms, it is a workflow that defines how software moves from code to production in an automated and repeatable manner.

Key Benefits of a Jenkins Pipeline

- Automates CI/CD processes
- Provides better visibility into the build process
- Offers version control of build logic (Pipeline as Code)
- Supports complex workflows (parallel execution, conditional logic, etc.)

Types of Jenkins Pipelines

There are two main types of syntax used to define Jenkins pipelines:

1. Declarative Pipeline	**2. Scripted Pipeline**
Simpler and more structured syntax Designed for users who want an easy-to-read format Introduced later to make pipelines more readable and maintainable Key Features: • Uses pipeline {} block • Organized into stages and steps • Easier error handling • Better validation in the Jenkins UI	More flexible and powerful Written in Groovy-based syntax Gives full control over the build process Suitable for advanced users Key Features: • Uses node {} block • Everything is written as code • Allows complex scripting and logic • Less user-friendly for beginners

2.2 Declarative Pipeline — Creation

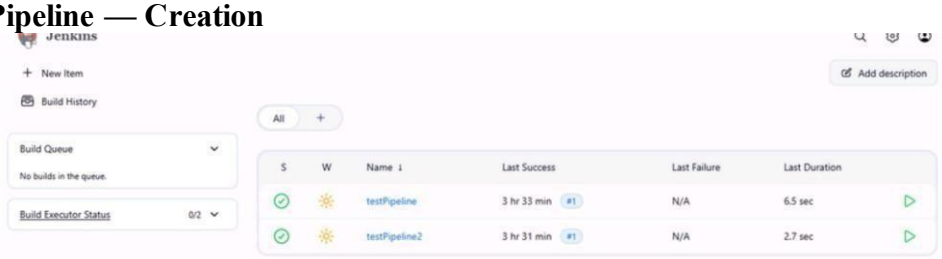


Fig. 10 — Jenkins dashboard showing the created pipelines (testPipeline, testPipeline2)

Code

```
pipeline
{
   agent any

   stages                {
stage('Hello')          {
steps {                 echo
'Heenal'
          }
```

```
    }
  }
```

```
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\JenkinsHome\workspace\Pipeline2
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Hello)
[Pipeline] echo
Ishita
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
}
```

Output

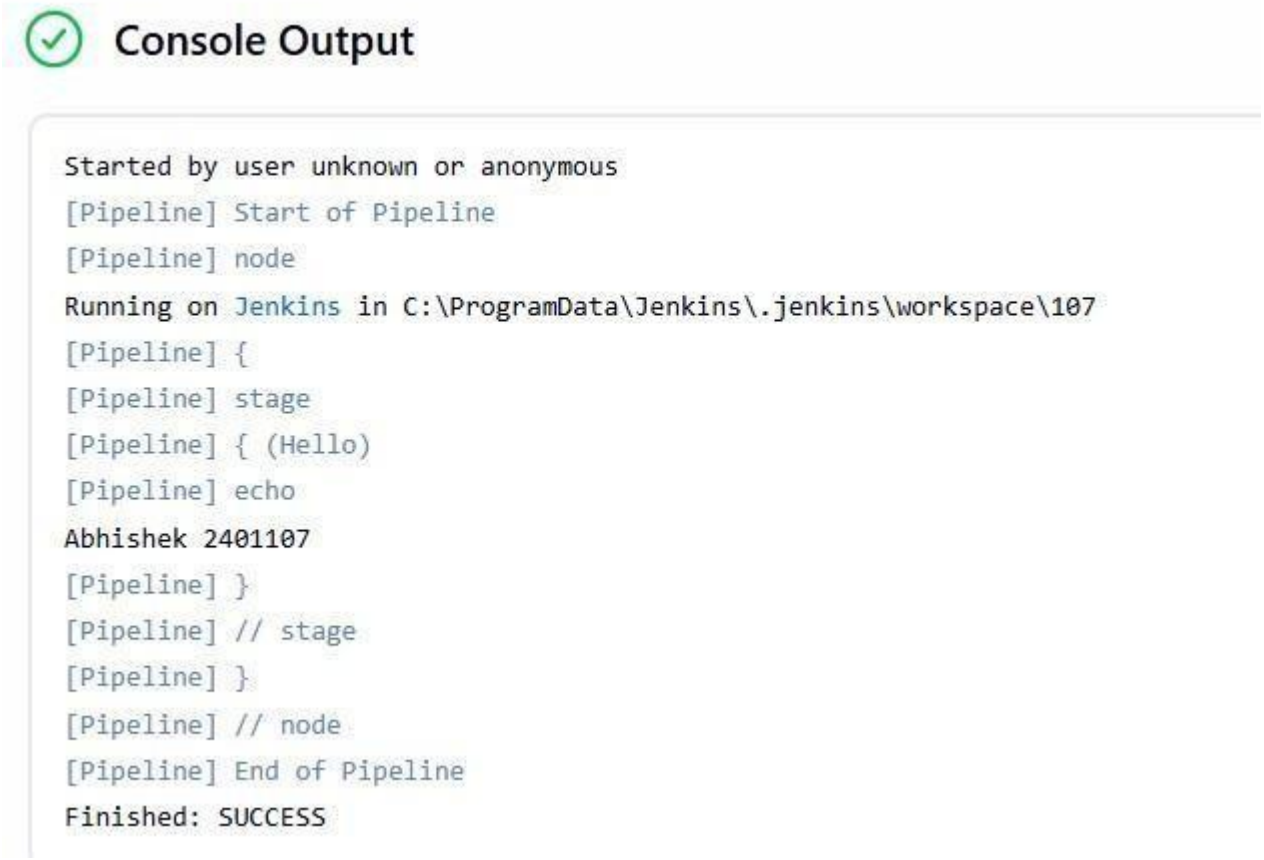


Fig. 11 — Console output confirming successful pipeline execution

2.3 Scripted Pipeline — Creation

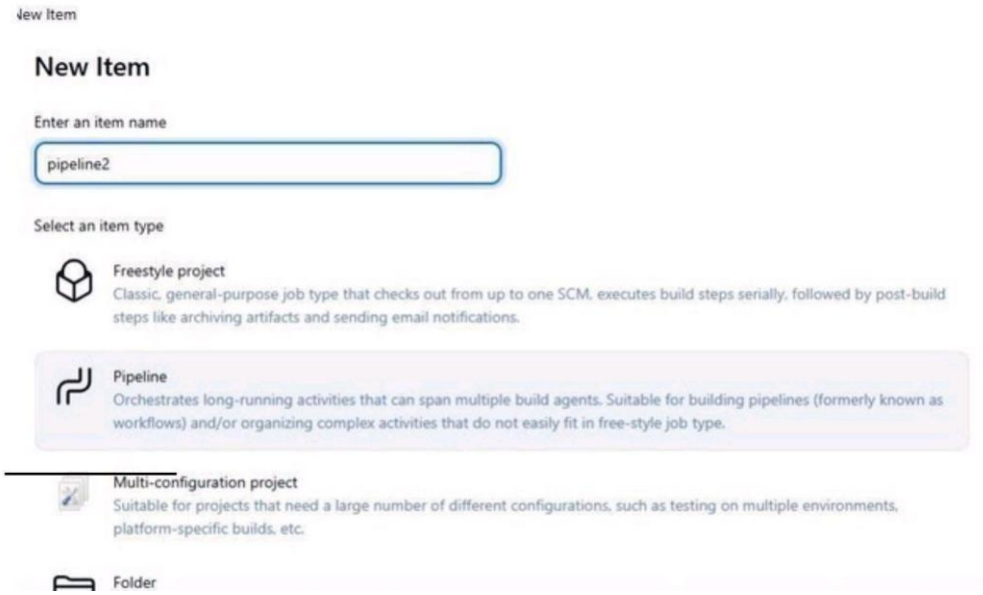


Fig. 12 — Creating a new Pipeline item ("pipeline2") in Jenkins

CONCLUSION: LO1, LO3

In this assignment, we learned how to create Jenkins Pipelines to automate the build, test, and deployment process. We explored both Declarative and Scripted pipelines, understanding their structure, benefits, and use cases. By implementing different pipeline examples, we demonstrated how Jenkins enables efficient, repeatable, and reliable CI/CD workflows.